Integración de Servicios Cloud GCP

Docker desde cero: desplegar, imagen y contenedor

Semana 14 - Sesión 27

Rel Guzman



Revisión: lo que vimos en la Sesión 26

Balanceador de carga

Recibe a todos los clientes en una sola dirección y los reparte entre varios servidores iguales.

CDN

Guarda cerca del cliente una copia de los archivos que son iguales para todos.

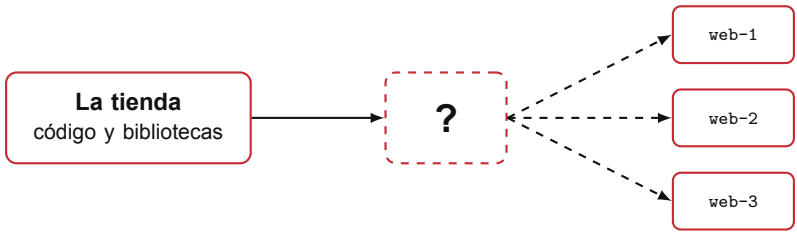
Contenedor

Un paquete con tu programa y todo lo que necesita para funcionar.

Responde

1. Con 3 servidores y round robin, ¿a cuál va el cliente 5?
2. Si `web-2` no responde, ¿qué hace el balanceador?
3. ¿Por qué el carrito de Ana no va en la CDN?

Revisión: la pregunta de hoy



1. Desplegar	2. Imagen	3. Contenedor
¿Por qué copiar el código al servidor casi nunca alcanza?	¿Cómo se lleva la app junto con todo lo que necesita?	¿Cómo se enciende, se detiene y se borra una copia?

Logro de la sesión

Al finalizar, habrás encendido en tu computadora una **API** hecha con **Python** y **Flask**, y podrás explicar con tus palabras qué es **desplegar**, qué es una **dependencia**, en qué se diferencian una **imagen** y un **contenedor** y con qué comandos se enciende, se detiene y se borra un contenedor.

Saberes previos

- En la Sesión 11, antes de encender el programa de Python, instalaste algo con `pip`. ¿Qué era y para qué servía?
- Cuando escribes `localhost:8080` en el navegador, ¿qué indica el 8080?
- Si cierras tu laptop, ¿alguien más puede usar un programa que tenías encendido en ella?

Utilidad de esta sesión

Para que cualquiera use una aplicación, hay que **desplegarla**: llevarla a un servidor y dejarla funcionando. Hoy empezarás encendiendo una API para ver qué necesita para funcionar. Después verás cómo Docker la lleva junto con todo eso en una **imagen** y cómo se manejan sus **contenedores**.

Práctica

La API de Café Nube, con Python y Flask en tu computadora

Práctica: la API de Café Nube

Caso de estudio

Café Nube, una cafetería de barrio, quiere mostrar su carta en internet. Primero necesita una **API**: un programa que no muestra páginas, sino que **entrega datos** a otros programas.

```python
import os
from flask import Flask, jsonify
app = Flask(__name__)

MENU = [
    {"nombre": "Americano", "precio": 8},
    {"nombre": "Capuchino", "precio": 10},
    {"nombre": "Mocaccino", "precio": 12},
]

@app.get("/menu")
def menu():
    return jsonify(MENU)

puerto = int(os.environ.get("PUERTO", 8080))
app.run(port=puerto)
```

Línea	En palabras
`from flask import ...`	Usa Flask, una biblioteca para hacer APIs
`MENU = [...]`	La carta: nombre y precio en soles
`@app.get("/menu")`	Si alguien pide `/menu`…
`return jsonify(MENU)`	…le responde la carta en JSON
`os.environ.get(...)`	Lee la variable `PUERTO`; si no existe, usa 8080
`app.run(port=puerto)`	Enciende la API y espera pedidos

JSON: texto con datos que otro programa puede leer:
`{"nombre":"Americano","precio":8}`

Práctica: preparar y encender

Windows (PowerShell)

```
# 1. Instala Python
winget install --id Python.Python.3.12 -e
# 2. Cierra y abre PowerShell, y comprueba
python --version
# 3. Enciende la API (prueba 1)
python api.py
# 4. Instala lo de requirements.txt
python -m pip install -r requirements.txt
# 5. Enciéndela otra vez
python api.py
```

Comando	Qué hace
`winget install`	Instala un programa en Windows desde PowerShell
`--id ... -e`	Elige el paquete con ese nombre exacto
`python api.py`	Ejecuta el programa `api.py`
`python -m pip install`	Instala las bibliotecas que pide `requirements.txt`

Comprueba en el navegador: `http://localhost:8080/menu`

```
[{"nombre":"Americano","precio":8},{"nombre":"Capuchino","precio":10},
 {"nombre":"Mocaccino","precio":12}]
```

Carpeta `Sesion27/practica_cafe_nube/`. La guía completa está en el `README.md`.

Práctica: seis pruebas

#	Qué hacer	Qué observar
1	Paso 3: enciende la API **antes** de instalar `requirements.txt`	¿Qué error sale? ¿Qué le falta a la API?
2	Abre `localhost:8080/menu` o, en otra terminal, `curl.exe localhost:8080/menu`	¿Qué responde? ¿Qué anota la terminal de la API?
3	Abre `localhost:8080/postres`	¿Qué código anota la terminal?
4	En otra terminal, enciende la API otra vez	¿Qué mensaje sale? ¿Por qué?
5	En esa otra terminal: `$env:PUERTO = "9090"` y luego `python api.py`	¿Ahora sí enciende? Abre `localhost:9090/menu`
6	Agrega un café a `MENU` y recarga `/menu`	¿Aparece? ¿Y después de reiniciar la API?

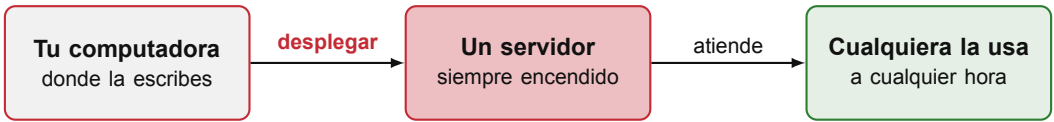
La idea

Además de su código, la API necesita **Python**, **Flask** (lo que dice `requirements.txt`) y **un puerto libre**. Reto en el `README.md`: la ruta `/menu/<nombre>`.

Docker desde cero

Desplegar, imagen y contenedor

¿Qué es desplegar?



Servidor

Una computadora **siempre encendida y conectada** a la red para atender a otros. Lo que la hace servidor es su trabajo.

Desplegar (*deploy*)

Instalar y poner en marcha una app donde la van a usar, **para que quede funcionando**.

La API de Café Nube corre en tu laptop: si la cierras, **nadie más** puede pedir la carta.

«En mi máquina funciona»: las dependencias



Dependencia

Todo lo que una app necesita para correr y **no es su propio código**: la versión del lenguaje, las bibliotecas, ciertos archivos del sistema.

Lo viste en la práctica

`ModuleNotFoundError: No module named 'flask'`: el código estaba completo, faltaba una dependencia.

Imagen y contenedor

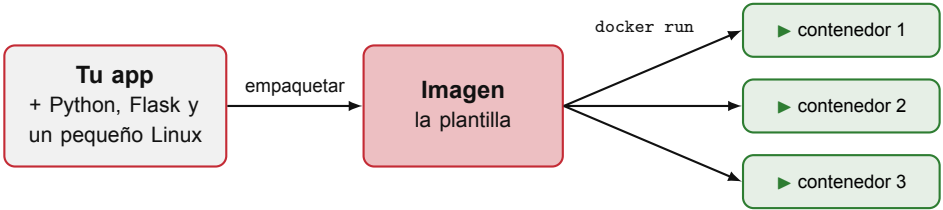


	Imagen	Contenedor
Qué es	Una plantilla: la app con sus dependencias	Una copia de la imagen **en marcha**
¿Corre?	Nunca	Mientras su programa trabaja
¿Cuántos?	Una	Todos los que quieras

Lo importante

Si Lucía le pasa a Diego la **imagen**, Diego solo necesita **Docker**: Python y Flask ya vienen adentro.

Tu primer contenedor: `docker run hello-world`

```
$ docker run hello-world
Unable to find image 'hello-world:latest' locally (1)
latest: Pulling from library/hello-world
Status: Downloaded newer image for hello-world:latest

Hello from Docker! (2)
This message shows that your installation
appears to be working correctly.
```

(1) No tenía la imagen: **la descarga**. Solo la primera vez.

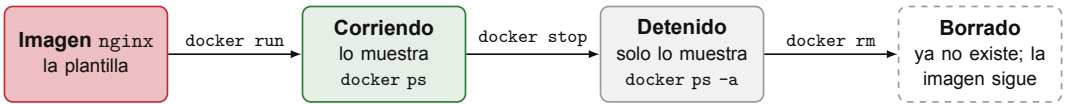
(2) Crea un contenedor, lo **arranca** y el programa escribe su saludo. Después termina.

`docker run` + una imagen

Crea un contenedor de esa imagen y lo arranca. Si la imagen no está en tu computadora, primero la descarga.

Antes, Docker Desktop debe estar abierto. Si responde `Cannot connect to the Docker daemon`, ábrelo y vuelve a intentar.

El ciclo de vida de un contenedor



Segundo plano

Una app de verdad no termina: queda esperando pedidos. Con `-d` corre **sin ocupar tu terminal**, y con `--name web` le pones nombre.

Logs

Los mensajes que escribe el programa mientras corre. Se leen con `docker logs web` y son **lo primero que miras** cuando algo falla.

Los comandos de todos los días

Comando	Qué hace	`web` queda
`docker run -d --name web nginx`	Lo crea y lo arranca en segundo plano	Corriendo
`docker ps`	Lista los que corren: aparece `web`	Corriendo
`docker logs web`	Muestra sus mensajes	Corriendo
`docker stop web`	Lo detiene, sin borrarlo	Detenido
`docker ps -a`	Lista también los detenidos: `Exited`	Detenido
`docker rm web`	Lo borra; la imagen `nginx` sigue	Borrado

Cuidado

`docker rm` no borra un contenedor que está corriendo: primero `docker stop`.

Cierre: preguntas para comprobar

Responde con tus palabras

1. En la práctica, ¿por qué falló la API antes de instalar `requirements.txt`?
2. ¿Por qué no pudiste encender dos veces la API en el puerto 8080? ¿Cómo lo resolviste sin cambiar el código?
3. Lucía copia la API a un servidor, pero no la enciende. ¿Ya la desplegó?
4. Lucía le pasa a Diego la **imagen** de la API. ¿Qué necesita Diego para correrla?
5. Detuviste `web` con `docker stop`. ¿Sigue existiendo? ¿Con qué comando lo ves?

Idea final

Una app no es solo su código: tu API necesita Python, Flask y un puerto libre. Una **imagen** lleva la app con todo eso, y cada **contenedor** es una copia en marcha que se enciende, se detiene y se borra.

Referencias

- Práctica: `github.com/rgap/rgap-Integracion-de-Servicios-Cloud-Gcp-Agosto2026`, carpeta `Sesion27/practica_cafe_nube/`.
- Python: `python.org`. Flask: `flask.palletsprojects.com`. winget: `learn.microsoft.com/windows/package-manager`.
- Docker y Docker Desktop: `docs.docker.com`.
- Material previo del curso: Sesión 11 (Python, `pip` y variables de entorno) y Sesión 26 (imagen y contenedor).
- Las respuestas de las preguntas están en `../respuestas.md`.